

计算机论文介绍—

JMStore: Joint Optimization of Computation and Storage Balancing in Multi-NDP Key-Value Stores with Hash-Based Data Distribution

论文来源: ACMTransactions on Computer Systems

一、研究背景与问题

随着全球数据量的爆炸式增长, IDC 预测到 2027 年全球数据总量将达到近 250ZB, 其中非结构化数据占比高达 86.8%。在这一背景下, 键值存储 (Key-Value Store) 凭借其简洁的数据模型和卓越的读写性能, 成为管理海量非结构化数据的主流方案。以 Log-Structured Merge-Tree (LSM-tree) 为基础的 KV 存储系统——如 LevelDB、RocksDB、Redis 和 Cassandra——被广泛应用于各类数据密集型场景。

然而, **LSM-tree 架构有个根本性缺陷**: Compaction 操作。LSM-tree 通过将随机写入转化为顺序写入来提升写性能, 但定期执行的 Compaction 操作需要合并重叠的 SSTable 文件, 这一过程涉及大量数据在存储设备与 CPU 之间的迁移。在传统的以 CPU 为中心的架构中, 这种数据迁移不仅消耗带宽和计算资源, 还导致严重的写放大问题, 严重影响系统吞吐量。

对此, 研究者们探索了多种优化路径, 来解决这一问题:

路径一是改进 LSM-tree 结构本身 (如 WiscKey、HashKV 通过键值分离减少写放大) ; **路径二**是利用新型存储介质 (如 NVM) 或加速芯片 (GPU、FPGA) 来提升性能 ; **路径三**是采用近数据处理 (Near-Data Processing, NDP) 架构, 将计算任务下沉到存储设备端执行。

其中, NDP 架构通过将 Compaction 操作在数据存储位置直接处理, 大幅减少了数据迁移开销。然而, 单个 NDP 设备的计算能力有限, 难以应对大规模 KV 数据处理的算力需求。MStore 虽尝试利用多 NDP 架构, 但其基于键范围的数据分布策略在面对大规模数据时, 仍面临硬件效率利用不足、负载不均衡等问题。

二、JMStore 的核心设计

针对上述挑战, 本文提出了 JMStore——一种基于哈希数据分布的多 NDP 键值存储系统, 旨在实现计算与存储的联合优化。JMStore 的核心设计包含以下几个层面:

First: Container-Member 数据组织方法

JMStore 提出了一种创新的 Container-Member (C-M) 数据布局。在该布局中, 主机端维护一个 Container-Meta-LSM-tree (C-mLSM-tree), 用于统一管理所有 NDP 设备上 SSTable 的元数据信息; 每个 NDP 设备则维护各自的 Member-Meta-LSM-tree (M-mLSM-tree)。在设备端, 多个 NDP 设备共同构成一个扩展的 CLSM-tree (Container-LSM-tree), 每个 CL 层由多个 ML 层组成。

这一设计的核心优势在于: 通过多 NDP 设备扩展 LSM-tree 每一层的宽度, 有效减少了树的深度, 从而降低了写放大。同时, CL0 层的 SSTable 通过哈希分区与主机端的 Bucket List 对应, 使得点查询可以通过哈希运算快速定位目标设备。

Second: 高效编程模型与接口

JMStore 设计了一套面向并行多 NDP 架构的编程模型, 包含跨层传输协议和三个核心接口: mNDP_Put、mNDP_Get 和 mNDP_Scan。

跨层传输协议解决了主机与 NDP 设备之间的异构性带来的语义鸿沟问题。协议模块为每个 NDP 设备分配独立的控制通道、写入通道 (含 4 个 NDP_Flush 通道和 1 个 NDP_Compaction 通道) 和读取通道, 实现了控制流、写数据流和读数据流的并行传输。数据采用统一的包格式传输, 并通过 SNAPPY 算法压缩以降低传输延迟。

mNDP_Put 接口优化了写入路径。数据先写入 Write-Ahead Log 和 MemTable, 当 MemTable 达到阈值后转换为 Immutable MemTable, 由 NDP_Slice 模块通过哈希取模运算将 KV 对分配到对应的 Bucket List 中。值得强调的是, NDP_Flush 和 NDP_Compaction 可以并行执行, 这是 JMStore 提升写性能的关键机制之一。

mNDP_Get 和 mNDP_Scan 接口则充分发挥了多 NDP 架构的并行查询能力。Get 操作首先查询 MemTable 和 Immutable MemTable, 然后通过补偿键缓存或哈希运算定位目标设备。为实现批量并行查询, JMStore 设计了批量并行查找机制, 将累积的查询请求按设备分组后多线程并行执行。Scan 操作则在每个 NDP 设备上创建独立迭代器并行扫描, 再将结果汇总排序后返回给用户。

Third: 存储资源均衡策略

JMStore 采用基于 Murmurhash 哈希函数的数据分片方案, 将用户数据均匀

分布到各 NDP 设备。然而, 真实工作负载往往存在显著的数据倾斜现象——大部分访问请求集中在少量热点数据上。为此, JMStore 设计了动态存储均衡策略——该策略的核心机制是哈希补偿 (Hash Compensation) : 在每个 NDP_Flush 操作后, JMStore 收集各设备的存储资源使用情况, 计算平均值。当某设备的使用量与平均值之差超过阈值 (默认 64MB) 时, 触发存储均衡, 利用虚拟 Bucket List 中的数据对存储不足的设备进行全量补偿, 并将补偿的键注册到补偿键缓存中。这一机制有效缓解了哈希倾斜工作负载下的数据分布不均问题。

Fourth: 计算资源均衡策略

在 Compaction 任务分配不均时, 慢速设备会成为整个系统的性能瓶颈。JMStore 提出了两种互补的 Compaction 任务均衡策略: 多线程并行回写 (MTPW) 策略针对 I/O 密集型的 Compaction 写回阶段。Compaction 操作分为三个阶段: 准备、读取合并、写回。其中写回阶段占 Compaction 总时间的大部分, 且计算负载较轻。MTPW 策略将待写回的数据块分组后分配给多个线程并行处理, 有效缩短了写回时间。

分层动态负载均衡 (HDLB) 策略针对计算密集型的场景。当某 NDP 设备的 Compaction 任务量显著超过阈值时, HDLB 策略通过设置 “Compaction 红灯” 标记, 跳过上层 (如 ML₁ 层) 的部分 SSTable 参与本次 Compaction。被跳过的 SSTable 保留在原层, 从而控制单次 Compaction 的数据量, 实现任务负载的动态调节。

两种策略根据设备间的计算能力差异和任务量差异自动选择: 差异较大时触发 HDLB, 差异适中时触发 MTPW。

三、实验验证与性能表现

JMStore 在配备 AMD Epyc 7763 (64 核) 主机和 36 个 NDP 设备的平台上进行了全面评估。每个 NDP 设备配备 4 核 Cortex-A55 处理器和 4GB 内存。实验采用 DB_Bench 和 YCSB-C 基准测试, 对比了单 NDP 的 PStore 和多 NDP 的 MStore。

在写密集型负载下, JMStore 的吞吐量最高可达 MStore 的 4.41 倍、PStore 的 13.85 倍。写延迟相比 MStore 降低 53.5%~77.2% , 相比 PStore 降低 84.9%~92.6% 。写放大相比 MStore 降低 57.1% , 相比 PStore 降低 77.19% 。Compaction 带宽相比 MStore 提升约 25% , 相比 PStore 的理论最大值提升

71.93%。

在混合读写负载下, JMStore 的优势更为显著——在 YCSB-C 测试中, JMStore 的性能提升可达 MStore 的 18.09 倍、PStore 的 92.57 倍。在扫描负载下, JMStore 分别为 MStore 和 PStore 的 12.55 倍和 6.19 倍。存储均衡策略将哈希倾斜工作负载下的设备间数据差异从 3.6GB 降至 1.4GB。计算均衡策略使吞吐量提升约 9.38%, 写延迟降低 8.57%, 写放大降低 10.49%~12.99%。

此外, 与无计算能力的多存储设备架构 (JMStore-) 相比, JMStore 的吞吐量最高可达前者的 6.3 倍。在实际数据集 OpenAlex 和 Amazon 上的测试也验证了 JMStore 的优越性。

四、在金融领域的潜在应用

金融行业面临的挑战: 金融行业正面临前所未有的数据压力。随着 AI、大数据技术在银行业的深度应用, 行业对软硬件设施的要求持续提升。七成受访银行表示, 现有数据库无法支撑半结构化、非结构化、向量等数据的处理需求。金融市场的交易数据、行情数据、客户行为数据等均呈现高并发、低延迟、海量非结构化的特征。

在具体场景中, 高频交易系统要求订单处理延迟从毫秒级降至微秒级; 风险控制需要实时分析海量交易网络关系; 智能投顾和个性化推荐系统则依赖对大规模用户行为数据的实时查询与更新。这些需求对底层数据存储系统提出了极高要求: 既要有极高的写入吞吐量 (应对持续涌入的交易数据), 又要有极低的读取延迟 (满足实时风控和交易决策), 还要能高效处理大规模非结构化数据。

对此 JMStore 的核心技术特性与金融行业的数据处理需求高度契合: JMStore 的在处理上述金融领域的挑战展现出如下核心竞争力: 1、**超高写入吞吐量, 支撑海量交易数据实时入库**。2、**极低读取延迟, 满足实时风控与交易决策**。3、**写放大显著降低, 延长存储设备寿命**。4、**计算与存储的联合优化, 适配金融级 SLA**。5、**优秀的扩展性, 应对金融数据规模的持续增长**。

应用场景一: 交易系统的高频数据存储。证券交易所、券商的高频交易系统每秒处理数十万笔订单。JMStore 可作为底层 KV 存储引擎, 利用多 NDP 设备并行处理订单数据的写入和查询, 将订单处理延迟从毫秒级降至微秒级。

应用场景二: 实时风控系统的特征存储与查询。风控模型需要实时查询交

易账户的历史行为、关联网络等特征数据。JMStore 的 mNDP_Get 接口支持批量并行查询,能够在亚毫秒级时间内完成数千个特征键的并行检索,为风控决策提供实时数据支撑。

应用场景三: 客户画像与个性化推荐。 金融机构需要基于海量用户行为数据构建 360 度客户画像。JMStore 的 mNDP_Scan 接口支持高效的并行范围扫描,能够快速检索特定用户群体的行为数据,支撑实时推荐和精准营销。